

1. 목적별 특화 프롬프트 설계 1

1. 목적별 특화 프롬프트 설계 전략

프롬프트 엔지니어링에서는 해결하려는 과제의 목적에 따라 프롬프트 작성 방법을 달리해야 합니다. 이번 세션에서는 크게 두 가지 목적 영역 (데이터 분석 vs. 보고서 작성)에 맞춘 프롬프트 설계 전략을 다룹니다.

1.1. 데이터 분석 목적 프롬프트 설계

데이터 분석을 목적으로 GPT 모델을 활용할 때는 데이터 요약, 코드 생성, 기초 통계 분석 등에 특화된 프롬프트를 작성합니다. 이러한 프롬프트는 분석 대상 데이터를 효율적으로 설명하고, 필요한 산출물을 명확히 요구하는 것이 핵심입니다.

- **데이터 요약/인사이트 추출:** 데이터셋의 특성을 한눈에 파악할 수 있도록, 전체적인 경향이나 그룹별 특징을 요약하도록 프롬프트를 설계합니다. 예를 들어 "주어진 표 데이터의 주요 특징을 요약하고 간단한 인사이트 3개를 bullet point로 알려줘"와 같이 요청할 수 있습니다. 이때 데이터의 컬럼명, 의미, 특이점 등을 미리 설명해 주면 모델이 더 나은 요약을 제공합니다.
- **시각화용 코드 생성:** GPT에게 **Python 코드** 생성을 요청하여 데이터 시각화를 도울 수 있습니다. 이 경우 프롬프트에 원하는 그래프 종류와 사용할 라이브러리 (예: matplotlib)를 명시합니다. 예를 들어 "matplotlib를 사용해 이 데이터의 분포를 보여주는 히스토그램 코드를 작성해줘"처럼 프롬프트를 주면, 모델이 해당 그래프를 그리는 코드를 생성합니다. 코드 프롬프트에서는 필요한 변수명/컬럼명을 정확히 지정하고, 가능하다면 주석이나 축 레이블 등도 포함하도록 요구합니다.
- **기초 통계 분석:** 평균, 중앙값, 분산 등 기본 통계량을 구하거나 그룹별 통계를 비교하는 작업도 프롬프트로 요청 가능합니다. 예를 들어 데이터셋에 대해 "각 컬럼의 평균과 중앙값을 계산해줘" 또는 "종류별로 레코드 개수를 알려줘" 같이 지시할 수 있습니다. 이 때도 필요한 경우 모델이 계산 방법이나 출력 형식(표 형태 등)을 따르게 구체적으로 서술합니다.

위와 같은 데이터 분석 프롬프트를 설계할 때는 **정확한 지시와 형식 요구**가 중요합니다. 또한 대화형 프롬프트일 경우 **시스템 메시지**나 **추가 예시**를 제공하여 모델이 답변할 때 분석가 역할을 충실히 따르도록 만드는 것도 한 방법입니다.

1.2. 보고서/문서 작성 목적 프롬프트 설계

GPT 모델은 마케팅 요약, 기술 문서 작성, 비즈니스 인사이트 도출 등 **텍스트 생성 중심** 작업에도 활용될 수 있습니다. 이 경우 프롬프트는 **문체, 대상 독자, 포맷** 등을 세심하게 지정하

여 원하는 스타일의 문서를 생성하도록 유도합니다.

- **마케팅 요약:** 마케팅 보고서나 캠페인 결과를 요약할 때는, **핵심 성과 지표(Key Metrics)**와 **인사이트**가 강조된 요약을 요청합니다. 예컨대 **"다음 마케팅 캠페인 결과를 간략히 요약하고, 주요 성과 3가지를 bullet point로 제시해줘"**처럼 프롬프트를 작성할 수 있습니다. 마케팅 맥락을 살려 **긍정적인 성과**와 **향후 권장 전략** 등을 포함하도록 모델에게 역할을 부여하면 더욱 설득력 있는 요약이 됩니다 (예: "당신은 마케팅 분석가입니다...").
- **기술 문서 작성:** 기술 문서나 요약본을 생성하려면 **정확성과 간결함**에 초점을 맞춰야 합니다. 모델에게 **"주어진 기술 설명을 바탕으로, 비전문가도 이해할 수 있도록 한 문단으로 요약해줘"** 혹은 **"기술 사양을 bullet 목록으로 정리해줘"**라고 지시할 수 있습니다. 이때 **전문 용어**는 필요한 경우에만 사용하고, **예시나 비유**를 통해 쉽게 풀어쓰도록 프롬프트에서 요구하면 효과적입니다. 또한 **"...에 대해 기술 문서 스타일로 자세히 작성해줘"**와 같이 문서의 형태 (예: 백서, 가이드 문서 등)를 지정할 수도 있습니다.
- **비즈니스 인사이트 도출:** 비즈니스 데이터를 요약하여 인사이트를 얻고자 할 때는, 단순 나열보다 **의미 해석**에 중점을 두어 프롬프트를 작성합니다. 예를 들어 **"지난 분기 판매 데이터에서 눈에 띄는 트렌드 3가지를 찾아 시사점을 설명해줘"**와 같이 요청하면, GPT는 증가/감소 추세나 상관관계 등 **임팩트 있는 포인트**를 추출해 줄 것입니다. 인사이트 프롬프트에서는 **"왜 이러한 결과가 나타났는지"** 또는 **"향후 무엇을 해야 하는지"**를 추가로 물어보도록 설계하면 모델이 분석을 조금 더 발전시켜 서술하게 만들 수 있습니다.

보고서 목적 프롬프트의 공통점은 **맥락 설정**과 **요구사항 명확화**입니다. 모델에게 **역할 부여** (예: "당신은 데이터 분석가/기술 전문가입니다..."), **대상 독자** (예: "경영진에게 보고하는 스타일로"), **형식** (예: "한글로 300자 이내 요약") 등을 상세히 알려주면 원하는 문체와 내용을 얻을 가능성이 높아집니다.

2. 실습 1: Playground – 프롬프트 템플릿 설계 및 스타일 비교

이제 간단한 **Playground 실습**을 통해, 동일한 입력이라도 프롬프트 설계에 따라 출력 스타일이 어떻게 달라지는지 확인해보겠습니다. Playground는 OpenAI 제공 인터페이스를 사용하여 바로 모델과 상호작용할 수 있는 환경입니다. 여기서는 목적별 프롬프트 **템플릿**을 직접 작성하고 결과를 비교해보는 연습을 합니다.

1. **샘플 텍스트 준비:** 먼저 요약/분석할 간단한 **예시 텍스트**를 제공합니다. 마케팅 분야 예시를 하나 들어보겠습니다. 아래는 가상의 마케팅 캠페인 결과 보고 텍스트입니다:

##실습 1.1. OpenAI Playground 실습

예시 텍스트 (마케팅 보고서 발췌):

XYZ사의 2024년 1분기 마케팅 캠페인 결과, 신규 사용자 5만 명 유치와 전년 동기 대비 매출 20% 증가를 달성했습니다. 주요 채널별 퍼포먼스는 다음과 같습니다:

- SNS 광고 캠페인: 클릭률 5%, 전환율 2%, 약 3만 명의 신규 방문자 유입
- 이메일 마케팅: 오픈율 30%, 클릭률 5%, 재참여 고객 10% 증가
- 검색 광고: 클릭률 8%, 전환율 3%, 매출 기여도 50%

이 결과를 바탕으로, 향후 캠페인에서는 SNS와 검색 광고에 대한 투자를 확대하고 이메일 전략을 개선할 계획입니다.

1. **프롬프트 템플릿 작성:** 이제 위 텍스트에 대해 서로 다른 요구사항을 가진 두 가지 프롬프트를 작성해봅시다. 첫 번째 프롬프트는 **마케팅 요약 인사이트**에 초점을 맞추고, 두 번째 프롬프트는 **경영진 보고용 요약**에 초점을 맞추어보겠습니다.

- **프롬프트 A (인사이트 강조 요약):** "다음 마케팅 보고서 내용을 한글로 요약해줘. 핵심 성과 지표와 인사이트 3가지를 bullet point로 알려줘."
- **프롬프트 B (경영진 보고 요약):** "당신은 마케팅 분석가입니다. 아래 결과를 바탕으로 경영진에게 보고할 간결한 요약본을 작성하세요. 주요 성과를 한 문단으로 정리하고, 끝에 앞으로의 제안 1-2줄을 덧붙이세요."

위 두 프롬프트는 동일한 입력 텍스트에 대해 **요구 형식과 어조가 다르기 때문에**, GPT-4o-mini 모델이 출력하는 결과도 달라질 것입니다. 프롬프트 A는 간결한 bullet 형식의 응답을 유도하며, 프롬프트 B는 문장으로 된 서술형 요약과 제안을 생성하도록 합니다.

2. **결과 비교:** Playground에서 각각의 프롬프트를 입력하고 GPT-4o-mini 모델로 실행하여 나온 결과를 살펴봅시다. 예상대로:

- 프롬프트 A의 출력은 예를 들면 다음과 같이 **주요 지표를 나열한 bullet 형태**가 될 수 있습니다:
 - 신규 사용자 5만 명 유치, 매출 20% 증가 (전년 동기 대비 달성)
 - SNS 광고와 검색 광고가 높은 전환으로 전체 성과에 크게 기여
 - 이메일 마케팅을 통해 재참여 고객 10% 증가 — 추후 이메일 전략 개선 여지 있음

(모델의 실제 출력 내용은 다를 수 있으나, 대체로 위와 같이 **숫자 중심의 핵심 성과**와 **인사이트 요약**이 나올 것입니다.)

- 프롬프트 B의 출력은 **하나의 요약 문단**으로 작성되어, **"XYZ사는 2024년 1분기 마케팅 캠페인에서 전년 대비 20%의 매출 성장을 이루고 5만 명의 신규 사용자를 확보했습니다. 특히 SNS와 검색 광고에서 두드러진 성과를 보였으며, 이메일 마케팅도 재참여율 10% 증가에 기여했습니다. 이러한 성과를 바탕으로 향후 SNS 및 검색 광고 투자를 늘리고 이메일 전략을 개선할 계획입니다."**와 같이 비교적 격식 있고 통합된 서술로 나타날 것입니다.

이처럼 **프롬프트 설계에 따라 같은 정보라도 출력 형식이** 크게 달라집니다. 수강생들은 Playground 실습을 통해 프롬프트를 변형해 보면서 원하는 스타일을 얻기 위한 표현 방법 (예: "bullet로 알려줘", "간결한 한 문단으로" 등)을 감각적으로 익힐 수 있습니다.

- 추가 연습 제안:** 위 마케팅 예시 외에도, 수강생들은 **기술 문서나 비즈니스 메모** 등의 샘플 텍스트로 유사한 실습을 해볼 수 있습니다. 예를 들어 기술 사양 문단을 주고 **"요점을 정리해 기술 블로그 글처럼 작성해줘"**라고 하거나, 비즈니스 회의 메모를 주고 **"중요 결정 사항 3가지만 뽑아 bullet으로 정리해줘"**라고 프롬프트를 바꿔보며 결과를 비교합니다. 이러한 연습을 통해 **목적과 대상에 최적화된 프롬프트 작성** 능력을 향상시킬 수 있습니다.

3. 실습 2: Colab – GPT와 데이터 분석 (요약 및 시각화 코드 생성)

이제 Colab에서 **실제 데이터셋을 다뤄보는 실습**을 진행합니다. 공개된 CSV 형식의 예제 데이터셋을 불러와서, GPT-4o-mini 모델을 활용해 데이터를 요약하거나 파이썬 코드를 생성해보겠습니다. 여기서는 **Iris 꽃 데이터셋 (iris.csv)**을 사용하지만, 수강생들은 Kaggle의 간단한 마케팅 데이터 (예: **은행 마케팅 캠페인 데이터**) 등으로 응용해 볼 수도 있습니다.

3.1. Colab 환경 및 OpenAI API 설정

Colab 노트북을 열고, 최신 OpenAI Python 라이브러리를 설치 및 임포트합니다. 그리고 Colab의 **userdata**를 통해 API 키를 설정합니다. 다음은 Colab에서의 설정 예시입니다:

##실습 1.2. Google Colab 실습

```
!sudo apt-get install -y fonts-nanum
# 폰트 설치 후 Colab 메뉴에서 런타임/세션 다시 시작을 선택합니다.

!pip install -U openai

from google.colab import userdata
import os
import openai
```

```
# Colab에 저장한 Secret에서 API 키 가져와 환경 변수에 등록
os.environ["OPENAI_API_KEY"] = userdata.get('OPENAI_API_KEY')

client = openai.OpenAI()
```

위와 같이 설정한 후 `client` 객체를 통해 모델에게 질문을 보낼 수 있습니다. (* 실제로 openai 라이브러리에서 `OpenAI()` 클래스와 사용 방법은 해당 버전의 공식 문서를 참고합니다.)

3.2. 데이터셋 불러오기 (iris.csv 예시)

분석에 사용할 예제 데이터셋을 Pandas를 통해 불러옵니다. iris 데이터는 꽃받침(sepal)과 꽃잎(petal)의 길이/너비 및 종(species) 정보를 담은 유명한 데이터셋입니다. Pandas로 CSV를 읽어들이어 DataFrame으로 저장해보겠습니다:

```
import pandas as pd

# 예제 iris 데이터셋 불러오기
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
df = pd.read_csv(url)
print(df.head(5))
```

위 코드는 인터넷에서 iris.csv를 직접 가져오는 예시입니다. (오프라인 환경에서는 제공된 `iris.csv` 파일을 업로드한 후 경로를 사용합니다.) `df.head(5)` 출력 결과로 데이터셋의 처음 5행을 확인할 수 있습니다:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

데이터셋이 성공적으로 `df`에 로드되었습니다. 이제 GPT 모델을 활용하여 이 데이터에 대한 요약과 시각화 코드 생성을 차례로 시도해보겠습니다.

3.3. GPT-4o-mini를 활용한 데이터 요약

우선 iris 데이터의 전반적인 특징을 GPT에게 요약해달라고 요청해보겠습니다. 프롬프트에는 데이터의 대략적인 내용(컬럼과 의미)을 설명한 후, 요약 형식으로 **간단한 통계 및 인사이트 3가지**를 달라고 해보겠습니다. 프롬프트 예시는 다음과 같습니다:

```
prompt_summary = f"""
다음은 Iris 꽃 데이터셋입니다. sepal_length, sepal_width, petal_length, petal_w
idth 컬럼으로
꽃의 크기 특성이 주어지고 species 컬럼으로 품종(setosa, versicolor, virginica)이
구분됩니다.

데이터:
{df.to_markdown()}

이 데이터의 주요 특성을 요약하고, 간단한 인사이트 3가지를 bullet point로 알려주세
요.
"""

response = client.chat.completions.create(
    messages=[
        {"role": "user", "content": prompt_summary}
    ],
    model="gpt-4o-mini"
)

summary_text = response.choices[0].message.content
print(summary_text)
```

프롬프트에서 **데이터 설명**과 **요구 사항**을 한국어로 명확히 전달했습니다. GPT-4o-mini 모델이 이 프롬프트를 처리하면, 예를 들어 다음과 같은 응답을 생성할 것입니다 (실제 응답 내용은 모델에 따라 달라질 수 있음):

- *Iris* 데이터셋은 세 가지 품종(*setosa*, *versicolor*, *virginica*)의 꽃 특성을 담고 있으며, 각 품종당 50개의 샘플로 총 150개 데이터로 구성됩니다.
- 전반적으로 *petal_length*와 *petal_width*는 품종에 따라 차이가 뚜렷하며, *setosa* 품종이 다른 두 품종에 비해 이들 값이 현저히 작습니다.
- *sepal_length*의 평균은 *versicolor*와 *virginica*가 *setosa*보다 크며, *virginica* 품종의 꽃이 전반적으로 가장 큰 경향을 보입니다.

위와 같이 모델은 데이터의 행/품종, 컬럼 간 특성 차이, 품종별 경향 등 **요약 정보**를 bullet 형태로 반환했습니다. 이러한 텍스트 요약은 보고서 작성이나 개략적 데이터 설명 파트에 활용될 수 있습니다.

(참고: 간단한 통계량 예시를 위해, `df.describe()` 나 `df.groupby('species').mean()` 등의 결과를 미리 확인한 후 주요 수치를 프롬프트에 포함하거나, 모델 응답과 비교해볼 수 있습니다.)

3.4. GPT-4o-mini를 활용한 시각화 코드 생성

다음으로, GPT에게 **matplotlib**를 이용한 **시각화 코드** 생성을 부탁해보겠습니다. Iris 데이터셋의 예시로 **"품종별로 sepal length와 sepal width의 분포를 보여주는 산점도 (scatter plot) 코드를 작성해줘"**라는 프롬프트를 줄 수 있습니다. 프롬프트 예시는 아래와 같습니다:

```
prompt_code = f"""
```

```
데이터:
```

```
{df.to_markdown()}
```

```
데이터를 활용하여 품종(species)별 꽃받침(sepal)의 길이와 너비 관계를 나타내는 산점도를 그리는
```

```
Python 코드를 작성해줘. matplotlib 라이브러리를 사용하고, 품종(species)에 따라 다른 색으로 표시하고
```

```
범례를 포함하도록 해줘.
```

```
"""
```

```
response = client.chat.completions.create(
    messages=[
        {"role": "user", "content": prompt_code}
    ],
    model="gpt-4o-mini"
)
```

```
code_text = response.choices[0].message.content
print(code_text)
```

모델이 생성한 `code_text` 는 Python 코드 형식의 문자열일 것입니다. 예를 들어 GPT-4o-mini는 다음과 같은 코드를 제안할 수 있습니다:

```
# GPT-4o-mini가 생성한 예시 코드
import matplotlib.pyplot as plt

# 품종별로 색상 매핑
colors = {'setosa': 'red', 'versicolor': 'green', 'virginica': 'blue'}

# 산점도 그리기
plt.figure(figsize=(10, 6))
for species, color in colors.items():
    subset = df[df['species'] == species]
    plt.scatter(subset['sepal_length'], subset['sepal_width'],
                color=color, label=species, alpha=0.6)

# 제목과 레이블 추가
plt.title('Sepal Length vs Sepal Width by Species')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.legend()
plt.grid(True)

# 그래프를 화면에 표시
plt.show()
```

Colab에 해당 코드를 입력하고 실행하면, iris 데이터의 품종별 꽃받침 길이/너비를 비교하는 **산점도 그래프**가 출력됩니다.

위 그림은 iris 데이터셋의 Sepal Length(꽃받침 길이)와 Sepal Width(꽃받침 너비)를 품종별로 색상을 달리하여 시각화한 산점도입니다. 각 점은 한 개의 꽃을 나타내며, 붉은색은 setosa, 초록색은 versicolor, 파란색은 virginica 품종에 해당합니다.

모델이 생성한 코드를 실행한 결과를 통해, **GPT가 제안한 코드의 정확성과 그래프의 의미**를 함께 검토합니다. 필요한 경우 그래프 축 레이블이나 제목 등을 수정하거나, 다른 종류의 시각화(예: 히스토그램, 박스플롯 등)를 추가로 시도해볼 수 있습니다.

Tip: GPT에게 코드를 요청할 때 필요한 부분만 실행하도록 프롬프트를 조정할 수 있습니다. 예를 들어 "... 코드만 답변하고 설명은 하지 마" 또는 "... 코드 블록으로만 답변해"라고 지시하면, 불필요한 해설

없이 코드 출력만 얻을 수 있습니다. 이렇게 하면 코드를 복사하여 바로 실행하기가 더 편리합니다.

3.5. 결과 활용 및 추가 분석

위 실습을 통해 얻은 **요약 텍스트**와 **생성된 코드**는 이후 단계에 다양하게 활용될 수 있습니다. 몇 가지 확장 아이디어는 다음과 같습니다:

- **요약 텍스트 저장/활용:** GPT가 출력한 데이터 요약 파일을 저장하거나 보고서에 바로 인용할 수 있습니다. Colab에서 `with open('summary.txt','w') as f: f.write(summary_text)` 처럼 파일로 저장해두면 나중에 다시 참고하거나 편집할 수 있습니다. 또한 해당 요약이 실제로 정확한지를 수강생들이 검증해보는 것도 추천됩니다 (예: 모델이 잘못된 내용을 썼는지 원 데이터와 비교).
- **코드 수정 및 재활용:** 모델이 생성한 시각화 코드를 수동으로 개선해보는 것도 좋은 실습입니다. 예를 들어 그래프 스타일을 꾸미거나 (`plt.style.use`), seaborn 라이브러리로 바꿔 그려보는 등의 추가 작업을 해볼 수 있습니다. 모델에게 다시 **"위 코드에서 X 기능을 추가해줘"**라고 후속 프롬프트를 보내 코드를 개선시키는 **프롬프트 체이닝(prompt chaining)**도 가능하며, 이는 더욱 복잡한 자동화에 응용될 수 있습니다.
- **GPT-4o 모델 사용:** 만약 GPT-4o-mini의 응답이 부족하거나 추가 성능이 필요하다면, `model="gpt-4o"` 로 변경하여 보다 강력한 GPT-4o 모델을 사용할 수 있습니다. GPT-4o는 더 높은 맥락 이해 능력과 정확도를 보여주므로, 특히 **복잡한 분석 요약**이나 **긴 문서 생성**에 유리합니다. (단, 호출 속도나 비용 측면에서 GPT-4o-mini보다 부담이 될 수 있으므로 적절한 경우에만 활용합니다.)

4. 출력 형식별 프롬프트 설계 방법 (JSON, 표, 목록)

프롬프트 엔지니어링에서는 원하는 **출력 형식**을 사전에 지정함으로써, 모델이 더 구조화된 응답을 내놓도록 유도할 수 있습니다. 특히 JSON, 표(tabular format), 목록(list) 형식은 **데이터 후처리나 정리**를 위해 많이 활용됩니다. 각 형식별 프롬프트 작성 팁은 다음과 같습니다:

- **JSON 형식 출력:** JSON은 키-값 쌍의 구조화된 데이터 형태입니다. 모델에게 JSON 출력을 요구할 때는 **정확한 키 이름과 값 형태**를 함께 명시하는 것이 좋습니다. 예를 들어, iris 데이터 요약을 JSON으로 받고 싶다면 프롬프트에 **"다음과 같은 JSON 형식으로 출력해줘: {"species": ..., "count": ..., "sepal_length_avg": ...}"**처럼 원하는 키 구조를 제시합니다. 또한 **"JSON 이외의 다른 설명은 일절 하지 마"**라고 덧붙여 **불필요한 텍스트를 배제**시키는 것도 중요합니다. 이렇게 하면 모델이 오로지 JSON 객체만 생성하게 되고, 후에 `json.loads()` 로 파싱하기 수월해집니다. (OpenAI API의 함수 호출 기능이나

`response_format` 옵션을 사용할 수도 있지만, 프롬프트만으로 제어하는 연습을 해봅시다.)

- **표 형식 출력:** 표 형태의 응답이 필요하다면 **마크다운 표** 또는 CSV 형식을 고려할 수 있습니다. 마크다운 표는 편리하게 결과를 보여줄 수 있고, CSV 형식은 나중에 Pandas로 읽기 용이합니다. 예를 들어 **"품종별 통계를 표로 보여줘. 컬럼은 Species, Count, SepalLengthMean, SepalWidthMean 으로 하고 값들을 채워줘"**라고 하면, 모델이 다음과 같은 표를 그릴 수 있습니다:

Species	Count	SepalLengthMean	SepalWidthMean
setosa	50	5.0	3.4
versicolor	50	5.9	2.8
virginica	50	6.5	3.0

위와 같은 표를 받았다면, 그대로 보고서에 활용하거나 복사해서 스프레드시트에 붙여넣을 수 있습니다. 혹은 CSV 형태로 달라고 했다면 콤마로 구분된 줄이 출력되므로, 이를 `pd.read_csv()` 로 읽어서 활용할 수도 있습니다. 표 출력 프롬프트의 요령은 **열 헤더 이름을 미리 지정**하고 **"표 이외에 다른 말은 하지 말라"**고 제한하는 것입니다.

- **목록 형식 출력:** 불릿 포인트() 목록이나 번호 매기기(`1.`) 목록은 **아이디어 열거, 단계별 지시** 등에 유용합니다. 프롬프트에서 **"~를 bullet 형식으로 나열해줘"** 또는 **"~를 1, 2, 3 단계로 설명해줘"**라고 직접 지시합니다. 모델은 해당 형식을 따라 결과를 구성하는데, 이때 각 항목을 간결하게 쓰도록 유도하면 좋습니다. 예를 들어 **"데이터셋 준비부터 모델 훈련까지의 과정을 5단계 bullet 목록으로 설명해줘"**라고 하면 번호가 붙은 목록을 얻을 수 있습니다. 목록 출력은 특별한 파싱이 필요 없지만, **항목 수를 제한하거나 원하는 경우 글머리 기호까지 지정**할 수 있음을 기억합니다.
- **형식 지정 종합 Tip:** 프롬프트 예시를 직접 보여주는 것도 강력한 방법입니다. 예를 들어 JSON으로 출력받고 싶을 때 프롬프트에 작은 예를 들거나, 대화형으로 **"형식 예:"**를 제시하면 모델이 그 형식을 학습하여 따릅니다. 다만, 예시를 넣으면 토큰 소모가 늘어나니 중요 키워드를 나열하는 것으로 충분할 때도 있습니다. 또한 Chat API를 사용할 경우 **시스템 메시지**에 형식 요구사항을 넣거나, **함수 호출 기능**을 사용하면 보다 안정적으로 형식화된 출력을 얻을 수 있습니다.

위 방법들을 활용하면, 모델의 출력을 우리에게 **보다 유용한 형태로** 바로 얻을 수 있습니다. 이는 추후 후속 작업(예: JSON 파싱하여 데이터프레임 생성, 표를 문서에 그대로 붙이기 등)을 수월하게 해줍니다.

5. Playground 실습 2: 다양한 문서의 요약 및 문서화

이 절에서는 다시 Playground로 돌아가서, **다양한 유형의 글**에 대해 GPT 모델을 활용해보는 추가 실습을 안내합니다. 마케팅, 기술, 비즈니스 세 분야의 예시 입력을 간략히 다루고, 각 경우에 맞는 프롬프트를 설계해 보겠습니다.

##실습 1.3. OpenAI Playground 실습

- **마케팅 자료 요약:** 앞서 사용한 마케팅 캠페인 결과 텍스트를 다시 떠올려 봅시다. 이번에는 단순 요약이 아니라 **마케팅 문서화** 형태로 프롬프트를 바꾸어 볼 수 있습니다. 예를 들어, **"위 캠페인 결과를 바탕으로 내부 팀원들에게 보낼 5문장 분량의 요약 이메일을 작성해줘."** 라고 하면, 모델은 격식은 덜하지만 **명확한 사실 전달**이 되는 이메일 형식의 답변을 생성할 것입니다. 이를 통해 같은 정보라도 **커뮤니케이션 상대에 맞게 어조와 상세함을 조절**하는 프롬프트 설계 연습을 할 수 있습니다.
- **기술 문서 요약/작성:** 기술 관련 글(예: 논문 일부, 기술 블로그 초안)을 GPT로 요약하거나 다듬게 할 수 있습니다. 예시로, 간단한 기술 설명 문단(**"CLIP모델의 이미지에 대한 텍스트 생성 원리를 설명해줘."**)을 주고 **"위 내용을 기반으로 FAQ 형식으로 Q&A 3개를 만들어줘"**라고 프롬프트를 만들어볼 수 있습니다. 또는 **"이 기술 개념을 초보자도 이해할 수 있게 비유를 들어 설명해줘"**와 같이 요청하면, **복잡한 내용을 평이한 언어로 재구성**하는 결과를 얻을 수 있습니다. Playground에서 이러한 시도를 해보며, 프롬프트 어투에 따라 얼마나 전문적으로 또는 쉽게 설명이 바뀌는지 비교합니다.
- **비즈니스 인사이트 도출:** 비즈니스 시나리오 예로, 어떤 회사의 분기별 판매 데이터 요약 문장을 입력한 뒤 **"이 결과에서 추출할 수 있는 비즈니스 인사이트 3가지를 알려줘"**라고 할 수 있습니다. 모델은 단순한 사실 나열을 넘어, **의미 해석**이 가미된 인사이트를 제시하려 할 것입니다 (예: 시장 트렌드 변화, 특정 상품군의 성공 등). 또 다른 연습으로 **"~를 기반으로 SWOT 분석 결과를 표로 만들어줘"** 같은 고차원 요청을 던져볼 수도 있습니다. 이러한 실습을 통해 GPT가 **경영 컨설턴트**처럼 사고하도록 프롬프트를 조율하는 방법도 경험하게 됩니다.

각 수강생들은 Playground를 활용해 위와 같은 시나리오별 프롬프트를 직접 실험해보고, 얻어진 결과물을 서로 공유/비교하면 좋습니다. 이를 통해 **프롬프트 엔지니어링의 미묘한 차이**가 출력에 어떤 영향을 주는지 이해하고, 더 나은 프롬프트를 위한 아이디어를 얻을 수 있습니다.

6. Pandas DataFrame으로 GPT 출력 결과 다루기 (고급)

마지막으로, GPT의 응답을 **Pandas DataFrame으로 변환**하거나 **후처리**하는 방법을 간단히 소개합니다. 이는 필수 단계는 아니지만, 모델 출력이 구조화되어 있을 경우 데이터 분석 파이프라인에 통합하기 유용합니다.

- **JSON 응답 파싱:** 앞서 JSON 형식 출력을 요청한 경우, Python에서 이를 다루기는 간단합니다. 예를 들어 `summary_json = response.choices[0].message.content`에 **JSON 문자열**

들어있다면, Python 내장 `json` 라이브러리로 파싱하여 딕셔너리나 리스트로 변환할 수 있습니다:

```
import json
data = json.loads(summary_json)
print(type(data))
# 출력: <class 'list'> 또는 <class 'dict'>
```

JSON 최상위가 리스트라면 `data` 는 리스트이고, 딕셔너리라면 딕셔너리가 됩니다. 이 `data` 를 곧바로 Pandas DataFrame으로 만들 수도 있습니다:

```
import pandas as pd
df_summary = pd.DataFrame(data)
print(df_summary.head())
```

이렇게 하면 GPT가 생성한 JSON 내용이 표 형태로 DataFrame에 담기게 됩니다. 예컨대 각 품종별 통계 리스트를 받았다면 행렬 형태의 데이터프레임으로 바뀌어, 이후 그 래프로 그리거나 CSV로 저장하는 등의 작업이 수월해집니다.

- **표/CSV 응답 파싱:** 만약 GPT 응답을 CSV 문자열로 받은 경우 (`response_text` 가 `'Species,Count,AvgLength,...\nsetosa,50,5.1,...\n...'` 같은 형태), `pd.read_csv(StringIO(response_text))` 등을 이용해 바로 DataFrame으로 읽어올 수 있습니다. 반면 마크다운 표 형태로 받았다면, 조금 후처리가 필요합니다 (예를 들어 줄바꿈과 구분자 `|` 를 이용해 파싱). 실습 시간에 여유가 있다면, GPT에게 아예 CSV로 출력하게 지시하고 그것을 DataFrame으로 읽는 과정도 시연해 볼 수 있습니다.
- **모델 응답 정리 및 검증:** DataFrame으로 만들지 않더라도, 모델의 텍스트 응답을 코드로 검증/정리하는 기법도 소개할 만합니다. 예를 들어 GPT가 낸 통계 요약이 맞는지 Python으로 직접 계산한 값과 비교하는 코드를 작성해 보는 것입니다. 이는 모델의 한계나 오류를 파악하는 데 도움이 되고, 궁극적으로 더 정확한 프롬프트를 작성하는 피드백으로 이어집니다.

위 내용은 선택적인 고급 주제이며, 핵심은 **GPT의 결과를 다른 도구와 연계하여 활용할 수 있다**는 점이며, 이를 통해 완전히 자동화된 데이터 분석 리포트 작성 흐름도 구현해볼 수 있다는 것입니다.

7. 정리

프롬프트 엔지니어링 심화 주제로 목적별 프롬프트 설계와 다양한 실습을 진행했습니다.

GPT-4o-mini의 빠르고 비용 효율적인 특성을 활용하여, 데이터 분석과 보고서 작성이라는 두 가지 상반된 목적에 대응하는 프롬프트 디자인을 살펴보았습니다. Playground 실습을

통해 프롬프트 문구 조정만으로도 출력 형식과 내용이 어떻게 바뀌는지 체험했고, Colab 환경에서 실제 데이터셋을 다뤄보며 GPT를 데이터 사이언스 보조 도구로 활용하는 방법을 연습했습니다. 특히 JSON이나 표처럼 구조화된 출력을 얻는 기법과, 코드 생성/시각화 결과 검증 등 **Human-LLM 협업**의 요소들도 경험했습니다.

- 잘 설계된 프롬프트는 모델의 능력을 극대화하여, **원하는 형태의 정보**를 정확하게 뽑아낼 수 있다.
- 목적과 대상에 따라 **프롬프트의 어조와 지시사항**을 바꾸면, 같은 입력 데이터도 전혀 다른 결과물을 만들 수 있다.
- GPT를 데이터 분석에 활용할 때는 **요약 + 코드생성** 조합으로 사람의 노력을 절약할 수 있지만, 결과에 대한 **검증과 해석은 인간의 몫**으로 남겨두어야 한다.
- 출력 형식을 지정하면 후속 작업이 쉬워지며, 이를 위해 **명시적인 형식 지시와 예시 제공**을 활용한다.
- 프롬프트 엔지니어링은 모델과의 협업 과정이므로, **반복 실험과 튜닝**을 통해 최적의 지시를 찾아가는 것이 중요하다.